CS2610: Computer Organization and Architecture Lab

Lab 10: 2 Stage Pipelined processor

6th April 2026

Objective

In this lab, you will be working with a 2 Stage pipelined RISC-V core design shared in moodle. The aim of this assignment is to understand this design, implement a new instruction in it and implement an optimization on top of it.

1 Question 1 (20 Marks)

Explain how the following instructions will be executed, by detailing what happens in Stage1 and Stage2

1. Conditional branches : BEQ
2. Unconditional branch : JAL
3. ALU : ADD
4. LOAD instruction : LW
5. STORE instructions : SW

2 Question 2 (40 Marks)

This question needs you to implement operand forwarding in the pipelined design shared with you. Steps to be followed are :

- Understand the modules within the design, especially the interface and control signals.
- Ideate what kind of information
- Test out different scenarios to verify your implementation.

The following is how CHATGPT gives hints for this part, use it if you want :

2.1 Operand Forwarding in a Two-Stage Pipeline

We consider a two-stage in-order pipeline consisting of:

- **S1:** Instruction Fetch, Decode, and Register Read
- **S2:** Execute and Writeback

2.1.1 Data Hazard Scenario

In this pipeline, operands are read in S1, while results are produced and written back in S2. This leads to a potential *Read After Write (RAW)* hazard when consecutive instructions have data dependencies.

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
```

The execution timeline without forwarding is shown below:

Cycle	I1	I2
1	S1	
2	S2	S1 (reads stale R1)
3		S2

Instruction $I2$ reads register $R1$ in S1 during cycle 2, but $I1$ produces the correct value only at the end of S2 in the same cycle. Thus, $I2$ observes a stale value.

2.1.2 Operand Forwarding Mechanism

To resolve this hazard without stalling, operand forwarding (bypassing) is employed. The result computed in S2 is directly forwarded to the operand inputs of S1 in the same cycle.

- Forwarding path: **S2 $\rightarrow$ S1 (same-cycle bypass)**
- Avoids waiting for register file writeback

2.1.3 Hardware Implementation

Dependency Detection A hazard is detected by comparing:

- Destination register of instruction in S2 (rd_{S2})
- Source registers of instruction in S1 ($rs1_{S1}, rs2_{S1}$)

Forwarding Logic For each operand, a multiplexer selects between the register file value and the forwarded value:

$$\text{if } rd_{S2} = rs1_{S1} \Rightarrow \text{forward to operand 1}$$
$$\text{if } rd_{S2} = rs2_{S1} \Rightarrow \text{forward to operand 2}$$

Datapath Support Multiplexers are added at the inputs of the execute stage to select forwarded data when required.

2.1.4 Timing Considerations

Correct operation of same-cycle forwarding requires:

- The result in S2 must be available early in the cycle
- Operand selection in S1 must occur late in the cycle

If these timing constraints are not met, a pipeline stall may still be required.

2.1.5 Limitations

Forwarding may not eliminate all hazards. For example, in load-like operations where data is not available early in S2:

```
I1: LOAD R1, [R2]
I2: ADD R3, R1, R4
```

the value may arrive too late to be forwarded in the same cycle, necessitating a stall.

2.1.6 Summary

Operand forwarding in a two-stage pipeline eliminates RAW hazards by bypassing results from S2 to S1 within the same cycle. This improves pipeline efficiency while requiring modest additional hardware for dependency detection and operand selection.

3 Question 3 (40 Marks)

This part of the question involves you implementing M extension instructions, especially mul variants

Instruction	Name	Type	Opcode	funct3	funct7	Operation
mul	MUL Low	R	0110011	0x0	0x01	$rd = (rs1 \times rs2)[31:0]$
mulh	MUL High (S,S)	R	0110011	0x1	0x01	$rd = (rs1 \times rs2)[63:32]$
mulsu	MUL High (S,U)	R	0110011	0x2	0x01	$rd = (rs1 \times rs2)[63:32]$
mulu	MUL High (U,U)	R	0110011	0x3	0x01	$rd = (rs1 \times rs2)[63:32]$

Table 1: RISC-V Multiply Instructions

4 Evaluation and Submission Instructions

- You have to verify the implementation with small testcases
- Implement a counter that counts number of cycles till the last instruction, this will give the cycle count for the sample program.
- For Q2, you have to show the cycles improved for your own sample testcase
- We will release a testcase involving some MUL instructions. Show baseline number of cycles, improvement with Q2 part, and recompile with mul instructions and show improvement in number of cycles.

1. The assignment must be completed individually.
2. This assignment is a take home one, viva will be held in next lab.
3. The following artifacts must be submitted:
    - Screenshots of all the modifications in each module.
    - Sample test code with the screenshot of this test code running.
    - The updated logisim circuit file.
4. Compress all files into a single ZIP archive and submit it on Moodle.